Informe Práctica 5B

Diseño de casos de prueba para ConjuntoOrdenado

Asignatura: Ingeniería del Software II
Práctica: 5B - Prueba de estructuras de datos
Alumno: Rodrigo Medrano Alonso
Curso: 2025/2026

1. Objetivo

El objetivo de esta práctica es probar la clase ConjuntoOrdenado, que implementa la interfaz IConjuntoOrdenado.

Para ello he diseñado una serie de pruebas unitarias usando JUnit. En primer lugar, he aplicado técnicas de caja negra, concretamente partición equivalente y análisis de valores límite. Después, una vez corregidos los errores encontrados, he comprobado la cobertura de las pruebas para verificar que también se cumplen los criterios de caja blanca.

2. Métodos que se han probado

La interfaz IConjuntoOrdenado define los siguientes métodos:

Método	Descripción
add(E elemento)	Añade un elemento en la posición que le corresponde según el orden natural.
get(int index)	Devuelve el elemento que se encuentra en la posición indicada.
remove(int index)	Elimina y devuelve el elemento situado en la posición indicada.
size()	Devuelve el número de elementos del conjunto.
clear()	Elimina todos los elementos del conjunto.

3. Diseño de casos de prueba de caja negra

Los casos de prueba se han diseñado a partir de la documentación de la interfaz, sin tener en cuenta inicialmente cómo estaba implementada la clase por dentro.

Para el diseño se han usado dos técnicas:

- Partición equivalente.
- Análisis de valores límite.

3.1. Partición equivalente

Para cada método se han identificado casos válidos e inválidos.

Método add

Caso	**Resultado esperado**
Añadir un elemento en un conjunto vacío	El elemento se añade correctamente y el método devuelve true.
Añadir un elemento menor que los existentes	El elemento queda colocado al principio.
Añadir un elemento intermedio	El elemento queda colocado en su posición correcta.
Añadir un elemento mayor que los existentes	El elemento queda colocado al final.
Añadir un elemento repetido	El método devuelve false y el conjunto no cambia.
Añadir null	Se lanza NullPointerException.

Método get

Caso	**Resultado esperado**
Usar un índice válido	Devuelve el elemento correspondiente.
Usar un índice negativo	Lanza IndexOutOfBoundsException.
Usar un índice igual al tamaño del conjunto	Lanza IndexOutOfBoundsException.
Usar get sobre un conjunto vacío	Lanza IndexOutOfBoundsException.

Método remove

Caso	**Resultado esperado**
Eliminar el primer elemento	Devuelve y elimina el primer elemento.

Eliminar un elemento intermedio	Devuelve y elimina el elemento correcto.
Eliminar el último elemento	Devuelve y elimina el último elemento.
Usar un índice negativo	Lanza IndexOutOfBoundsException.
Usar un índice igual al tamaño del conjunto	Lanza IndexOutOfBoundsException.
Usar remove sobre un conjunto vacío	Lanza IndexOutOfBoundsException.

Métodos size y clear

Caso	**Resultado esperado**
Conjunto recién creado	size() devuelve 0.
Después de añadir elementos	size() devuelve el número correcto de elementos.
Después de eliminar elementos	size() se actualiza correctamente.
Después de ejecutar clear()	El conjunto queda vacío.
Ejecutar clear() sobre un conjunto vacío	No se produce ningún error.

4. Análisis de valores límite

El análisis de valores límite se ha aplicado principalmente en los métodos que trabajan con índices, es decir, get y remove.

Para un conjunto de tamaño n, los valores probados han sido:

Valor	**Tipo**	**Resultado esperado**
-1	Inválido	Excepción
0	Válido si el conjunto no está vacío	Primer elemento
Posición intermedia	Válida	Elemento correspondiente
n - 1	Válido	Último elemento
n	Inválido	Excepción

También se han probado los casos límite del método add, comprobando que los elementos se insertan correctamente al principio, en medio y al final del conjunto.

5. Casos de prueba implementados

En total se han implementado 16 pruebas unitarias con JUnit 5.

Estas pruebas cubren los siguientes aspectos:

- Creación de un conjunto vacío.
- Inserción de elementos.
- Comprobación de que se mantiene el orden natural.
- Comprobación de que no se permiten elementos duplicados.
- Comprobación de que no se permite añadir null.
- Acceso a elementos con índices válidos e inválidos.
- Eliminación de elementos en primera, media y última posición.
- Eliminación usando índices inválidos.
- Comprobación del tamaño del conjunto.
- Vaciado del conjunto con clear.
- Uso del conjunto después de haberlo vaciado.

6. Ejecución inicial sobre la versión base

Primero se ejecutaron las pruebas sobre la versión original de ConjuntoOrdenado, sin corregir previamente el código.

El comando utilizado fue:

mvn clean test

El resultado obtenido fue:

Tests run: 16, Failures: 3, Errors: 0, Skipped: 0
BUILD FAILURE

Los tests que fallaron fueron:

addMantieneOrdenNaturalAscendenteEnInicioMedioYFinal
addDuplicadoDevuelveFalseYNoModificaElConjunto
removeEnPrimeraMediaYUltimaPosicion

Estos fallos permitieron localizar errores en el método add.

El primer fallo indica que los elementos no se estaban guardando en orden natural ascendente. El segundo fallo indica que se permitían elementos duplicados, aunque la estructura debe comportarse como un conjunto. El tercer fallo aparece en una prueba de remove, pero realmente está causado por el orden incorrecto generado antes por add.

7. Corrección realizada

El método que tenía errores era add.

En la versión original, la comparación usada para buscar la posición de inserción era:

elemento.compareTo(lista.get(indice)) < 0

Esta condición hacía que los elementos se colocasen en orden incorrecto.

Además, el método no comprobaba si el elemento ya existía en el conjunto, por lo que siempre insertaba el elemento y siempre devolvía true.

La corrección consistió en:

1. Cambiar la comparación para mantener el orden ascendente.
2. Comprobar si el elemento ya existe antes de insertarlo.
3. Devolver false si el elemento está duplicado.

8. Ejecución final sobre la versión corregida

Después de corregir el método add, se volvieron a ejecutar las pruebas unitarias sobre el proyecto final.

El comando utilizado fue:

mvn clean test

El resultado fue:

Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Con este resultado se comprueba que la versión corregida supera todos los casos de prueba diseñados.

9. Cobertura de caja blanca

Para comprobar la cobertura requerida en las pruebas de caja blanca se utilizó JaCoCo.

El informe generado mostró los siguientes resultados:

Métrica	**Resultado**
Cobertura de instrucciones	100 %
Cobertura de ramas	100 %
Líneas cubiertas	17/17
Métodos cubiertos	6/6
Clases cubiertas	1/1

Por tanto, se consiguió:

Cobertura de sentencias: 100 %
Cobertura de decisión/condición: 100 %

Como la cobertura obtenida fue completa, no fue necesario añadir más casos de prueba después del análisis de caja blanca.

10. Conclusión

En esta práctica se ha diseñado una batería de 16 pruebas unitarias aplicando partición equivalente y análisis de valores límite.

La ejecución inicial sobre la versión base permitió detectar errores en el método add, concretamente en la ordenación de los elementos y en la gestión de duplicados.

Después de corregir estos defectos, todas las pruebas pasaron correctamente. Además, el análisis de cobertura confirmó que se alcanzó un 100 % de cobertura tanto en sentencias como en ramas.

Por tanto, considero que los casos de prueba diseñados son suficientes para validar el comportamiento de la clase ConjuntoOrdenado según la especificación de la interfaz.